Helm 101

Kubernetes Package Management the Right Way

You’ll learn:

- What Helm is & why it exists
- Core concepts (Charts, Releases, Values, Templates)
- How to install & use Helm daily
- Templating basics (Go templates + Sprig)
- Chart structure & best practices
- Dependencies, upgrades, rollbacks
- Security, testing, and CI/CD tips

> Instructor: *Helm & Kubernetes practitioner*

What is Helm?

Helm is the **package manager for Kubernetes**.

- Packages are called **Charts** (think: apt/yum/homebrew for k8s)
- A **Chart** describes a deployable application: Deployments, Services, Ingress, CRDs, etc.
- A **Release** is a **deployed instance** of a Chart into a cluster/namespace.
- **Values** provide configuration for templated manifests.

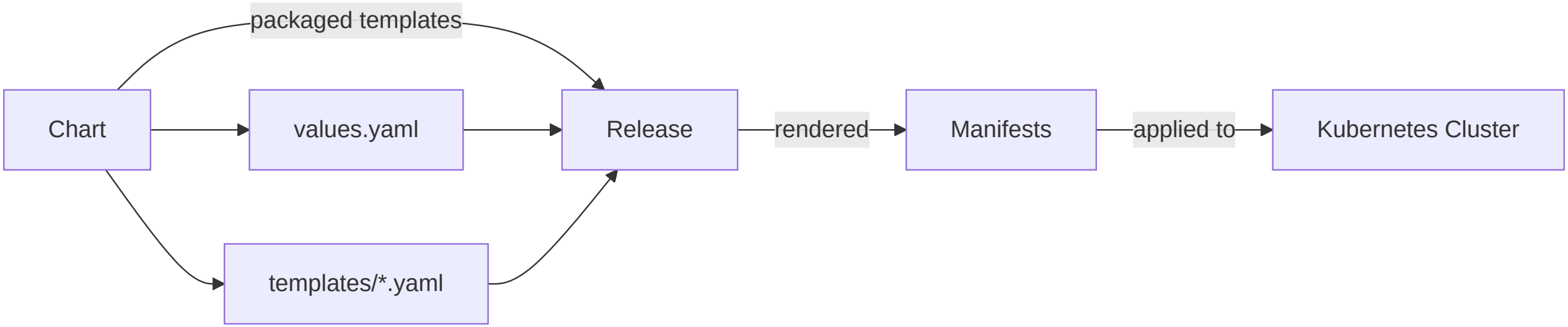
Why Helm?

- Reuse proven manifests as packages
- Parameterize deployments cleanly
- Version & share apps via repos / OCI registries
- Manage lifecycle: install/upgrade/rollback

Helm vs. Raw YAML vs. Kustomize

Feature	Raw YAML	Kustomize	Helm
Reusability	Low	Medium	**High**
Parameterization	Manual	Patches/vars	**Values + templates**
Packaging & Distribution	None	None	**Charts + Repos/OCI**
App Lifecycle (releases)	None	None	**Install/Upgrade/Rollback**
Dependency Mgmt	None	None	**Subcharts**

> You can combine Helm + Kustomize in advanced workflows, but Helm alone covers most app packaging needs.


packaged templates
Chart
values.yaml
Release
rendered
Manifests
applied to
Kubernetes Cluster
templates/*.yaml

Core Concepts (Mental Model)

- **Chart**: App definition + templates + metadata
- **Values**: Configuration inputs
- **Templates**: Go templates that render into Kubernetes YAML
- **Release**: Versioned instance in a namespace

Install Helm (Client)

```
# macOS (Homebrew)
brew install helm

# Linux (script)
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# Verify
helm version
```

> Server-side components (Tiller) were removed in Helm v3. It’s client-only now.

Quickstart: Use a Public Chart

```
# 1) Add a repo
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

# 2) Search charts
helm search repo nginx

# 3) Install a chart as a release
helm install web bitnami/nginx --namespace demo --create-namespace

# 4) Check status & resources
helm status web -n demo
kubectl get all -n demo

# 5) Uninstall
helm uninstall web -n demo
```

Chart Layout (Scaffold)

```
helm create myapp
```

```
myapp/
  Chart.yaml          # Metadata (name, version, apiVersion)
  values.yaml         # Default values
  charts/             # Subchart dependencies
  templates/          # Go-templated k8s manifests
    _helpers.tpl      # Named templates/partials
    deployment.yaml
    service.yaml
    ingress.yaml
    tests/test-connection.yaml
```

Minimal Chart Example

values.yaml

```yaml
author: "you"
image:
  repository: nginx
  tag: "1.27"
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
replicaCount: 2
```

templates/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "myapp.fullname" . }}
  labels:
    app.kubernetes.io/name: {{ include "myapp.name" . }}
    app.kubernetes.io/instance: {{ .Release.Name }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app.kubernetes.io/name: {{ include "myapp.name" . }}
      app.kubernetes.io/instance: {{ .Release.Name }}
  template:
    metadata:
      labels:
        app.kubernetes.io/name: {{ include "myapp.name" . }}
        app.kubernetes.io/instance: {{ .Release.Name }}
    spec:
      containers:
        - name: app
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - containerPort: {{ .Values.service.port }}
```

Chart.yaml Essentials

```yaml
apiVersion: v2
name: myapp
version: 0.1.0            # Chart version (SemVer)
appVersion: "1.0.0"      # App version displayed in status
kubeVersion: ">=1.24.0" # (optional) supported range
description: A demo chart
keywords: [demo, nginx]
type: application
```

Tips

- `version` controls chart packaging/versioning.
- `appVersion` is informational (container/app image version).
- Pin compatible Kubernetes versions via `kubeVersion`.

Rendering & Installing

```
# Render templates locally (no cluster change)
helm template myapp ./myapp -n demo

# Install
helm install myapp ./myapp -n demo --create-namespace

# Upgrade (with new values)
helm upgrade myapp ./myapp -n demo -f values-prod.yaml

# Roll back (to previous revision)
helm rollback myapp 1 -n demo
```

> `helm template` + `kubectl apply` is a useful pattern for GitOps or dry-run diffs.

Values: Precedence & Overrides

Order of precedence (highest → lowest):

1. `--set`, `--set-string`, `--set-file`
2. `-f custom-values.yaml` (multiple -f merge left→right)
3. Chart `values.yaml`

Examples

```
# Set scalars quickly
helm install api ./chart -n demo \
  --set replicaCount=3,image.tag=1.2.3

# Preserve strings exactly
helm upgrade api ./chart -n demo \
  --set-string resources.limits.cpu="500m"

# Inject file contents as values
helm install api ./chart -n demo \
  --set-file config=./app-config.yaml
```

Templating Basics (Go Templates)

Common template commands:

- `{{ .Values.foo }}` access values
- `{{ default "8080" .Values.port }}` defaulting
- `{{ include "name" . }}` reuse named templates
- `{{ required "msg" .Values.critical }}` hard requirement
- Control flow: `if`, `with`, `range`
- Functions: **Sprig** + Helm helpers

Example

```
metadata:
  name: {{ include "myapp.fullname" . }}
  labels:
    {{- include "myapp.labels" . | nindent 4 }}
```

Named Templates & Helpers

templates/_helpers.tpl

```
{{- define "myapp.name" -}}
{{- .Chart.Name -}}
{{- end -}}

{{- define "myapp.fullname" -}}
{{- printf "%s-%s" .Release.Name .Chart.Name | trunc 63 | trimSuffix "-" -}}
{{- end -}}

{{- define "myapp.labels" -}}
app.kubernetes.io/name: {{ include "myapp.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
helm.sh/chart: {{ .Chart.Name }}-{{ .Chart.Version }}
{{- end -}}
```

> Use helpers to keep templates DRY and consistent.

Files, Capabilities, and Built-ins

- `.Release`: name, namespace, revision, service
- `.Chart`: name, version, appVersion
- `.Values`: merged user values
- `.Files.Get`, `.Files.GetBytes`: read files in chart
- `.Capabilities.KubeVersion` / `.Capabilities.APIVersions`: branch logic by cluster

Example (API Version Gate)

```
apiVersion: {{- if .Capabilities.APIVersions.Has "networking.k8s.io/v1" -}}
networking.k8s.io/v1
{{- else -}}
extensions/v1beta1
{{- end -}}
```

Dependencies (Subcharts)

Chart.yaml

```yaml
dependencies:
  - name: redis
    version: "~17.0.0"
    repository: "https://charts.bitnami.com/bitnami"
    condition: redis.enabled
```

```
# Pull in dependencies
helm dependency update ./myapp

# Configure subchart via top-level values
redis:
  enabled: true
  architecture: standalone
```

- `condition` allows toggling subcharts via values
- `import-values` can map subchart values into parent

Hooks (Lifecycle Events)

Hooks are special templates executed at chart lifecycle events.

Example annotation

```
metadata:
  annotations:
    "helm.sh/hook": pre-install,pre-upgrade
    "helm.sh/hook-weight": "-5"
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
```

Use cases: DB migrations, Jobs before/after an upgrade.

> Keep hooks idempotent; failed hooks fail the release.

Tests

Place tests in `templates/tests/` as Kubernetes Jobs/Pods with special labels.

Example

```
apiVersion: v1
kind: Pod
metadata:
  name: "{{ include "myapp.fullname" . }}-test"
  labels:
    helm.sh/chart: "{{ .Chart.Name }}-{{ .Chart.Version }}"
    helm.sh/hook: test
spec:
  containers:
    - name: curl
      image: curlimages/curl
      args: ["-f", "http://myapp:80/health"]
  restartPolicy: Never
```


Run tests:

Linting, Packaging, and Distribution

```
# Lint your chart
helm lint ./myapp

# Package chart (creates myapp-0.1.0.tgz)
helm package ./myapp

# Serve a simple local repo (index.yaml)
helm repo index .

# OCI registries (Helm v3)
helm registry login ghcr.io
helm chart save ./myapp oci://ghcr.io/your-org/myapp:0.1.0
helm chart push oci://ghcr.io/your-org/myapp:0.1.0
helm install myapp oci://ghcr.io/your-org/myapp --version 0.1.0 -n demo
```

Debugging & Diffs

```
# Dry-run + debug
helm upgrade myapp ./myapp -n demo --dry-run --debug

# Render only
helm template myapp ./myapp -n demo > rendered.yaml

# Popular plugin: diff
helm plugin install https://github.com/databus23/helm-diff
helm diff upgrade myapp ./myapp -n demo -f values.yaml -f values-prod.yaml
```

Tip: Commit rendered manifests for review in PRs when helpful.

Security & Secrets

- Prefer **external secret stores** (External Secrets Operator, CSI providers)
- If embedding secrets via values, encrypt with **SOPS** + `helm-secrets` plugin
- Use **least privilege** RBAC for CI/CD service accounts
- Avoid checking plain `values.yaml` with secrets into VCS

```
# Example: helm-secrets
helm plugin install https://github.com/jkroepke/helm-secrets

# Encrypted values
helm secrets enc values-secrets.yaml
helm upgrade --install myapp ./myapp -n demo \
  -f values.yaml -f secrets://values-secrets.yaml
```

Environments & Value Strategies

- Separate `values-{env}.yaml` (dev/stage/prod)
- Keep **only diffs** across env files
- Use `--set-string` for exact strings (e.g., CPU "500m")
- Prefer **immutable tags** (image digests) for reproducibility

```
helm upgrade --install api ./chart -n prod \
  -f values.yaml -f values-prod.yaml \
  --set image.tag=sha256:abcd1234
```

Observability & Rollbacks

```
# What changed?
helm history myapp -n demo
helm get values myapp -n demo
helm get manifest myapp -n demo

# Roll back fast
helm rollback myapp 3 -n demo
```

Integrate with dashboards (Grafana/Loki/ELK) to watch for regressions after upgrades.

CI/CD Patterns

- Lint on PR: `helm lint`, `helm template` on multiple Kubernetes versions
- Use **chart testing** tools (ct) for multi-cluster checks
- Push charts to **OCI** or ChartMuseum after tagging
- Deploy via GitHub Actions/GitLab + `helm upgrade --install`

Example (pseudo pipeline)

```
steps:
  - run: helm lint ./myapp
  - run: helm dependency update ./myapp
  - run: helm template myapp ./myapp --kube-version 1.28
  - run: helm package ./myapp
  - run: helm chart push oci://ghcr.io/org/myapp:$(CHART_VERSION)
  - run: helm upgrade --install myapp oci://ghcr.io/org/myapp \
          --version $(CHART_VERSION) -n demo -f values-demo.yaml
```

Troubleshooting Cheatsheet

- `--dry-run --debug` to see rendered output & hooks
- Confirm CRDs are installed before charts that need them
- Check **apiVersion** compatibility (use `.Capabilities` guards)
- Validate **selectors** match **labels** in Deployments/Services
- Use `kubectl describe` & pod logs for runtime failures
- When in doubt: `helm get manifest` and inspect the YAML

Where to Find Charts

- **Artifact Hub** (community charts)
- **Vendor repos** (Bitnami, Elastic, etc.)
- **Your org’s registry** (OCI preferred)

Search

```
helm search hub postgresql
helm search repo bitnami/postgresql
```

Best Practices Summary

- Keep charts small, focused, and documented
- DRY with `_helpers.tpl` and `include`
- Pin versions (charts & images), avoid `latest`
- Separate env-specific values; keep secrets out of Git
- Test with `helm lint` and `helm test`
- Use hooks sparingly & idempotently
- Prefer OCI registries for distribution

Hands-on Lab (10–15 min)

1. Create a chart and deploy

```
helm create hello
helm install hello ./hello -n lab --create-namespace
```

2. Change replicaCount to 3, **upgrade**, then **view diff**

```
helm plugin install https://github.com/databus23/helm-diff
helm diff upgrade hello ./hello -n lab --set replicaCount=3
helm upgrade hello ./hello -n lab --set replicaCount=3
```

3. **Rollback** to previous revision

```
helm history hello -n lab
helm rollback hello 1 -n lab
```

Key References

- Helm docs: templates, values, hooks, dependencies
- Sprig functions reference
- OCI registry workflows
- Artifact Hub for discovering charts

> Practice: open a chart you use at work, read through `values.yaml`, `_helpers.tpl`, and `templates/`. Explain it to a teammate.